KUBERNETES — ARCHITECTURE, OPERATIONS & REFERENCE V1

Declared desired state, reconciled continuously — and the settings that decide whether it stays up
workloads · networking · storage · scheduling & resources · probes · security · operations · upgrades · **depth marked** junior → staff

1 · WHAT KUBERNETES IS — THE RECONCILIATION MODEL, AND WHEN NOT TO USE IT

start here — one idea explains the whole system, and the honest answer on whether you need it

The definition

A **container orchestration platform** built around a single idea: you **declare the state you want** as API objects, and **controllers continuously work to make reality match**. You never tell Kubernetes to do something — you tell it what should be true, and it keeps making that true.

The reconciliation loop is the whole mental model. Every controller runs the same loop: observe actual state, compare with declared state, take one step to close the gap, repeat forever. A node dies and pods are rescheduled; a pod is deleted and the ReplicaSet creates another. **Nothing is event-driven in the way people assume** — it is a continuous convergence process, which is why it is resilient and why it is sometimes slow.

The consequences worth internalising

1	**Everything is an API object** stored in etcd. `kubectl`, controllers and your CI all speak the same API. There is no privileged back door
2	**Nothing you create by hand survives.** Delete a pod created by a Deployment and it comes back — because the declaration did not change. Fighting a controller always loses
3	**Failures are normal and expected.** Pods are cattle. Nodes are replaceable. Anything relying on a specific pod living forever is working against the design
4	**Extensibility is the point.** Custom resources plus a controller means *your* domain object gets the same reconciliation behaviour — this is what operators are

Use it when

- **Many services** that need to be deployed, scaled and connected independently
- **You need self-healing and rolling updates** without building them yourself
- **Workloads vary** — batch, services, cron, machine learning — and you want one scheduling layer
- **Multiple teams** sharing infrastructure with real isolation and quotas
- **You have, or will build, platform capability.** This is the actual precondition

Do not use it when — and this is worth saying plainly. Kubernetes is a **large, permanent operational commitment**. For a single application, a small team, or a workload that a managed container service or a few virtual machines would run happily, it adds a great deal of complexity for benefits you will not use. **"We might need to scale one day" is not a reason** — you can move later, and moving later is cheaper than operating a cluster you did not need.

The honest test: can you name three problems you have *today* that Kubernetes solves? If the list is speculative, a managed platform-as-a-service or serverless containers will serve you better and cost less attention.

On running databases here

It is entirely possible with StatefulSets and mature operators, and many organisations do it well. It is also **strictly harder** than a managed database, and the failure modes are ones your team must understand. **Do it when you have a specific reason**, not by default because everything else is on the cluster.

STAFF **The release cadence is an operational fact, not a footnote.** Kubernetes ships **roughly three minor releases a year**, each supported for about fourteen months. That means **you are always upgrading** — a cluster left alone for two years is unsupported and on a path with no easy exit. Budget for continuous upgrades from the start; it is the single largest ongoing cost people fail to plan for.

Recent changes worth knowing

In-place pod resize	CPU and memory can now be adjusted **without restarting the pod** — a genuine improvement for right-sizing long-running workloads
Traffic distribution	`PreferSameNode` and `PreferSameZone` keep service traffic local, reducing latency and cross-zone data charges
Gateway API	**GA and the strategic direction** for ingress. See panel 5
Ingress NGINX retiring	The project moved to best-effort maintenance and archival. **If you run it, plan a migration** — this affects a very large number of clusters
containerd 1.x ending	Recent releases are the last supporting it — move to 2.x before upgrading past them

Two removals that still surprise people. Dockershim was removed in 1.24 — the runtime is containerd or CRI-O, and Docker as a runtime is gone (images are unaffected, they are just OCI images). **PodSecurityPolicy was removed in 1.25** — replaced by Pod Security Admission. Material written before those dates teaches things that no longer exist.

2 · WORKLOAD OBJECTS — WHAT TO USE FOR WHAT

JUNIOR the objects · MID when each is correct · STAFF StatefulSet's real guarantees

Object	Purpose and behaviour
Pod	The smallest deployable unit — one or more containers sharing a network namespace and storage. **Ephemeral, and you should almost never create one directly**
ReplicaSet	Keeps *n* identical pods running. **You do not manage these** — Deployments do
Deployment	**The default for stateless services.** Manages ReplicaSets to give rolling updates and rollback. Pods are interchangeable and get random names
StatefulSet	For workloads needing **stable identity**. Pods get ordinal names that persist across restarts, each keeps its own volume, and start-up and shutdown are ordered
DaemonSet	**One pod per node** — log collectors, node agents, CNI components, monitoring. Automatically covers new nodes
Job	Runs to completion. Handles retries and parallelism
CronJob	Creates Jobs on a schedule. **Set history limits** or completed Jobs accumulate indefinitely

A CronJob is not a reliable scheduler. It can miss runs if the controller is unavailable past the deadline, and it can run twice under some conditions. **Make the job idempotent**, set `startingDeadlineSeconds`, and choose a `concurrencyPolicy` deliberately.

What a StatefulSet actually guarantees, precisely. Stable network identity, stable storage bound to each ordinal, and ordered rolling operations. **It does not make your application distributed, consistent or highly available** — a clustered database still needs to do that work itself. StatefulSet provides the primitives; the application provides the correctness.

Multi-container patterns

Init containers	Run to completion **before** app containers start, in order. For migrations, waiting on a dependency, or fetching configuration
Sidecar	A container running alongside the main one — proxy, log shipper, secret refresher. **Native sidecar support** (an init container with `restartPolicy: Always`) fixed the long-standing problem of sidecars not starting first and not shutting down last
Ambassador / adapter	A sidecar that proxies outbound connections, or reshapes output for a monitoring system

Native sidecars are worth adopting. Before them, a Job with a service-mesh sidecar would never complete because the sidecar kept running, and app containers could start before their proxy was ready. The native form starts before app containers, shuts down after them, and does not block Job completion.

Labels and selectors

- **Labels are how everything finds everything.** Services select pods by label; Deployments own pods by label. There is no other linkage
- **A Deployment's selector is immutable** — plan the label scheme before you ship
- **Adopt a consistent set** — the recommended `app.kubernetes.io/*` labels give you consistent querying and cost attribution
- **Annotations** carry non-identifying metadata; they cannot be selected on

3 · CONFIG & SECRETS

and a widespread misconception

Secrets are not encrypted by default — they are base64-encoded, which is not encryption. Anyone who can read the Secret object, or read etcd, sees the value. This is the most common security misunderstanding in Kubernetes, and it is worth stating without hedging.

Making Secrets actually secret

- **Enable encryption at rest** for etcd, ideally with a KMS provider so keys live outside the cluster
- **Restrict access with RBAC** — very few roles need to read Secrets, and `get secrets` in a namespace is equivalent to holding those credentials
- **Prefer an external secret store** — Vault, or a cloud secrets manager via the External Secrets Operator or the CSI secret store driver. The value then lives outside etcd entirely
- **Never commit Secrets to git.** If you want them in git, use Sealed Secrets or SOPS so what is committed is encrypted
- **Prefer workload identity over static credentials** — a pod assuming a cloud role removes the secret altogether, which is strictly better than protecting it

ConfigMaps and Secrets in practice

As env vars	Simple, and **values do not update** without a pod restart. The snapshot is taken at start
As volume mounts	**Updates propagate** to the file after a delay — but your application must notice and reload, which most do not
Immutable	Marking them immutable improves cluster performance at scale and prevents accidental edits. Roll a new object to change

The reliable pattern: change configuration, roll the pods. Include a hash of the ConfigMap in the pod template annotation, so any config change produces a new template and triggers a normal rolling update. You then get the same rollout safety and rollback for configuration as for code — rather than a silent partial update where some pods have the new value and some do not.

4 · ARCHITECTURE — THE CONTROL PLANE, THE NODES, AND WHAT HAPPENS WHEN YOU APPLY A MANIFEST

every component and its job, and the path from a YAML file to a running container

CONTROL PLANE — the desired state, and the controllers that pursue it

API SERVER — everything goes through it
- The only component that talks to etcd
- Authentication, then authorisation (RBAC), then admission
- Validates and persists every object
- Serves watches — how controllers learn of changes

Stateless, so run several behind a load balancer. Its availability is the cluster's control-plane availability.

etcd — the only stateful component
- Distributed key-value store holding all cluster state
- Raft consensus, so an odd number of members — three or five
- Extremely sensitive to disk latency and network jitter

Back it up, test the restore, and give it fast local disks. Losing etcd without a backup means rebuilding the cluster and every object in it from source.

CONTROLLER MANAGER — the reconciliation loops
- Runs dozens of controllers in one process: Deployment, ReplicaSet, Node, Job, EndpointSlice, ServiceAccount and many more
- Each watches objects and works to close the gap to desired state
- The cloud controller manager handles provider integration — load balancers, node lifecycle, routes

This is where "Kubernetes does something" actually happens.

SCHEDULER — placement only
- Watches for pods with no node assigned
- **Filters** nodes that cannot run it — insufficient resources, taints not tolerated, affinity rules unmet
- **Scores** the survivors and picks the best, then writes the binding

It only decides where. It never starts anything — and it schedules on requests, not on actual usage. See panel 7.

ADMISSION CONTROL — the extension point that matters most for governance

Between authorisation and persistence, every request passes through admission plugins and webhooks. **Mutating** webhooks change the object — injecting a sidecar, adding default labels. **Validating** webhooks accept or reject it — enforcing that images come from an approved registry, that resource requests are set, that no pod runs as root.

This is how policy is enforced cluster-wide — Kyverno, Gatekeeper, or the built-in Validating Admission Policy. It is also a real availability risk: a webhook that is down or slow can block every write to the cluster. Set a failure policy deliberately, exclude critical namespaces, and give webhooks tight timeouts.

WORKER NODE — where containers actually run

KUBELET
- The node agent. Watches for pods bound to its node
- Instructs the runtime to pull images and start containers
- **Runs the probes** and reports pod status back
- Mounts volumes; enforces resource limits via cgroups

Evicts pods under node pressure — memory, disk, PIDs. Eviction order depends on QoS class. See panel 7.

KUBE-PROXY / CNI DATAPLANE
- Implements Service networking on the node
- Programs iptables, IPVS or eBPF rules so a ClusterIP resolves to a healthy backing pod
- Modern CNIs such as Cilium can replace it with eBPF

Service load balancing happens here, on every node — there is no central proxy in the data path.

CONTAINER RUNTIME
- containerd or CRI-O, spoken to via the CRI interface
- Pulls images, creates containers, manages their lifecycle
- **Dockershim was removed in 1.24** — Docker is no longer a runtime, though Docker-built images work unchanged

Recent releases are the last to support containerd 1.x — move to 2.x before upgrading past them.

FROM MANIFEST TO RUNNING CONTAINER — the path, and where each step can stall

1 kubectl apply sends to the API server → **2** authn, authz, admission webhooks → **3** persisted to etcd → **4** the Deployment controller sees it and creates a ReplicaSet → **5** the ReplicaSet controller creates Pod objects, unscheduled → **6** the scheduler filters and scores nodes, writes a binding → **7** the kubelet on that node sees it → **8** the runtime pulls the image and starts containers → **9** probes pass → **10** the pod becomes Ready and is added to Service endpoints.

Pending means step 6 failed — no node fits. ContainerCreating means step 8 — usually an image pull or a volume mount. CrashLoopBackOff means the container starts and exits.

READING THE DIAGRAM — six properties that explain the rest of this sheet

1 · **Controllers reconcile continuously; they do not execute commands.** This is why the system heals itself, why deleting a managed pod does nothing lasting, and why changes are eventually rather than immediately consistent.

2 · **The API server is the single gateway and etcd is the single source of truth.** Control-plane availability is API server availability; durability is etcd's backup. Everything else — kubelets, controllers, your CI — is a client.

3 · **The scheduler places pods using requests, not actual usage.** A node can be scheduled to capacity while sitting idle, or be overcommitted and start evicting. Requests are the most consequential number you write — see panel 7.

4 · **Admission control is where cluster-wide policy lives** — and it sits in the write path for every object. It is the right place to enforce standards, and a misconfigured webhook can block all writes, so treat its failure policy and timeouts as availability settings.

5 · **Service networking is distributed to every node.** There is no central proxy in the data path — each node programs rules locally. This scales well and means network debugging happens on the node, not at a chokepoint.

6 · **The kubelet, not the control plane, decides to evict under pressure.** Node-level resource exhaustion is handled locally and by QoS class — which is why a pod with no resource requests is the first thing killed on a busy node.

Useful interrogation of any cluster: is etcd backed up and has a restore been tested, do all workloads set resource requests, what is the pod security and network policy default, how are upgrades performed, and what happens if an admission webhook goes down?

Those five questions separate a cluster that is operated from one that is merely running — and the first and last are the ones most often answered with a pause.

On managed control planes: EKS, GKE and AKS run the API server, etcd, scheduler and controller manager for you, with backups and upgrades handled. You still own nodes, workloads, networking policy, security posture and cost. **This removes the hardest component to operate well and none of the ones you interact with daily** — which is why self-managing the control plane needs a specific justification.

5 · NETWORKING — SERVICES, DNS, AND INGRESS VERSUS GATEWAY API

MID the Service types · STAFF what is changing at the edge, and what is not

The model in one line: every pod gets a routable IP, and any pod can reach any other pod without NAT. That flat network is provided by the CNI plugin, and it is why Kubernetes networking feels simple until you need to restrict it.

Service type	Behaviour
ClusterIP	**The default.** A stable virtual IP reachable only inside the cluster, load-balancing across ready pods
NodePort	Opens a port on every node. Mostly a building block for LoadBalancer — rarely the right thing to expose directly
LoadBalancer	Provisions a cloud load balancer. **One per Service, which becomes expensive** — this is the main reason ingress exists
ExternalName	A DNS CNAME to something outside the cluster. No proxying
Headless clusterIP: None	**No virtual IP** — DNS returns pod addresses directly. What StatefulSets use so each replica is individually addressable

DNS and endpoints

- **Every Service gets a DNS name** — `svc.namespace.svc.cluster.local`, shortened to `svc` within the namespace
- **EndpointSlices** track which pods are ready and back the Service. **Only Ready pods receive traffic** — which is what makes readiness probes matter
- **CoreDNS is a critical dependency.** If it is unhealthy, everything appears broken in confusing ways. Give it resources, run enough replicas, and monitor it

Getting traffic in: Ingress and Gateway API

Ingress	The long-standing HTTP routing API. Simple, and its limitations pushed every implementation into **vendor-specific annotations** — so Ingress manifests are not really portable
Gateway API	**GA and the strategic direction.** A richer, role-oriented model: infrastructure teams own `GatewayClass` and `Gateway`, application teams own `HTTPRoute`. Native support for traffic splitting, header matching and cross-namespace routing — expressed in the API rather than in annotations

The nuance that matters: Ingress is not being removed. It is GA, still supported, and clusters running it are not on a deadline. **Gateway API is where new capability is going**, and it is the right choice for new work — but this is a migration to plan, not an emergency.

One thing *is* urgent, though. The widely used **Ingress NGINX controller has moved to best-effort maintenance and archival**. A very large number of clusters run it. If yours does, plan a move — to Gateway API with a maintained implementation, or to another actively developed controller. **This is the single most actionable item on this panel.**

Network policy

- **The default is allow-all.** Without policies, any pod can reach any other pod in the cluster, across namespaces. People consistently assume otherwise
- **Start with a default-deny ingress policy per namespace**, then allow what is needed. This is the highest-value security change available
- **Policies are enforced by the CNI — and not every CNI implements them.** Verify yours does before relying on it
- **Do not forget egress** — restricting outbound traffic limits what a compromised pod can reach

6 · STORAGE

and the access-mode misconception

Object	What it is
Volume	Storage attached to a pod. `emptyDir` lives and dies with the pod; `configMap` and `secret` project data in
PersistentVolume	A piece of storage in the cluster, usually provisioned dynamically rather than created by hand
PersistentVolumeClaim	**What a pod actually references** — a request for storage of a size and class
StorageClass	Defines *how* storage is provisioned — the disk type, parameters and reclaim behaviour
CSI driver	The interface storage vendors implement. Everything modern goes through it

The access-mode misconception. `ReadWriteOnce` means the volume is mounted read-write by **one node**, not one pod — several pods on the same node can share it. `ReadWriteMany` allows many nodes and **requires a filesystem that supports it**, such as NFS or a cloud file service. **Most block storage is ReadWriteOnce only**, which is why a Deployment with several replicas sharing one block volume will not schedule.

Behaviour worth knowing

- **Reclaim policy** — `Delete` destroys the underlying disk when the claim goes; `Retain` keeps it. **Check the default of your StorageClass** before deleting anything you care about
- **StatefulSet volumes are deliberately not deleted** when the StatefulSet is. This is a safety feature and a common source of orphaned disks and unexpected cost
- **Expansion** is supported by most drivers — grow the claim; shrinking is not possible
- **Volumes are usually zone-bound.** A pod with a claim in one zone cannot be scheduled to another — which quietly constrains rescheduling during an incident
- **`VolumeAttributesClass`**, now stable, allows changing volume parameters such as IOPS after provisioning

Prefer managed services for databases and object storage for blobs. Running stateful workloads on Kubernetes is entirely achievable with good operators, and it is **strictly more work** than the managed alternative. Make it a deliberate decision with a stated reason rather than a default.

7 · RESOURCES AND SCHEDULING — REQUESTS, LIMITS AND WHY PODS GET KILLED

the most consequential numbers in any manifest · STAFF the CPU-limit debate

The distinction everything rests on. `requests` are for scheduling — the scheduler uses them to decide whether a pod fits on a node, and it never looks at actual usage. **`limits` are for enforcement** — the kubelet caps the container at runtime. They are different numbers doing different jobs, and conflating them causes both wasted money and unexplained restarts.

CPU	**Compressible.** Exceeding the limit causes **throttling** — the container is slowed, not killed
Memory	**Not compressible.** Exceeding the limit means the container is **OOM-killed immediately**. There is no graceful degradation

Quality of service, which decides who dies first

Class	When, and eviction order
Guaranteed	Requests equal limits for every resource. **Evicted last.** Use for anything critical
Burstable	Requests set, lower than limits. The common case. Evicted after BestEffort
BestEffort	**No requests or limits at all. Evicted first**, and scheduled as though it needs nothing — so it lands on already-full nodes

Which gives the rule: always set requests. A pod without them is invisible to the scheduler, gets placed on saturated nodes, and is the first thing killed under pressure. **The great majority of "Kubernetes randomly killed my pod" reports are missing resource requests.**

STAFF **The CPU-limit debate, stated fairly.** Setting a CPU limit causes throttling **even when the node has idle CPU** — the container is held at its ceiling regardless. Many practitioners therefore recommend **setting CPU requests but omitting CPU limits**, while **always setting memory limits** (since memory cannot be reclaimed by throttling). The counter-argument is that unlimited CPU allows a noisy neighbour to degrade others. **Both positions are defensible**; what matters is choosing one deliberately and knowing that latency-sensitive services throttled by a low CPU limit is a very common and very confusing performance problem.

Placement controls

nodeSelector node affinity	Restrict pods to nodes with certain labels. Affinity adds soft preferences and richer expressions
Pod affinity and anti-affinity	Place pods near or away from other pods. **Anti-affinity is how you stop all replicas landing on one node**
Taints and tolerations	The inverse — a node **repels** pods unless they tolerate the taint. How you reserve nodes for specific workloads, such as GPU hosts
Topology spread	**The modern and better tool** for spreading replicas evenly across zones or nodes. Prefer it to anti-affinity, which is harder to express and slower to evaluate
PriorityClass	Higher-priority pods can **preempt** lower-priority ones when capacity is short. Powerful and easy to misuse — a high priority everywhere means no priority

Right-size from measurement, not from guesses. Use the Vertical Pod Autoscaler in recommendation mode, or actual usage percentiles, to set requests. **In-place pod resize now allows adjusting CPU and memory without a restart**, which makes this a far less disruptive exercise than it used to be.

8 · AUTOSCALING

three axes, easily confused

Scaler	Scales
HPA horizontal pod	**The number of pod replicas**, on CPU, memory or custom metrics. The one you will use most
VPA vertical pod	**The requests and limits of a pod.** Historically required a restart to apply — in-place resize is changing that. Excellent in recommendation-only mode
Cluster Autoscaler	**The number of nodes**, when pods cannot be scheduled. Works within node groups you define
Karpenter	Provisions nodes directly, choosing instance types to fit pending pods. **Faster and usually cheaper** than node-group scaling, at the cost of being another component
KEDA	Event-driven scaling from queue depth, stream lag, or almost any external signal — **and it can scale to zero**, which the HPA alone cannot

Do not run HPA and VPA on the same resource for the same workload. They will fight — VPA raises requests, which lowers CPU utilisation percentage, which makes HPA scale in, which raises utilisation again. Use VPA for memory and HPA for CPU, or VPA in recommendation mode only.

Making autoscaling actually work

- **Requests must be set** — the HPA computes utilisation as a percentage *of the request*. No request means no scaling
- **The metrics server must be running** for CPU and memory metrics. Its absence is the usual reason an HPA reports unknown targets
- **Tune stabilisation windows** so the workload does not flap. Scaling in aggressively then back out is worse than scaling slowly
- **Node scaling takes minutes**, pod scaling takes seconds. **Keep headroom**, or use low-priority placeholder pods that get preempted, so a traffic spike does not wait for a node to boot
- **Scale on the signal that reflects load** — queue depth or in-flight requests are often far better proxies than CPU

Scaling exposes what was already fragile. A service that cannot start quickly, or that holds a database connection per replica, will fail under autoscaling in ways it never did at fixed size. **Test scaling deliberately** rather than discovering it during a spike.

9 · PROBES & LIFECYCLE

the most misconfigured feature

Probe	What failure does
Liveness	**Restarts the container.** Only for genuinely unrecoverable states — a deadlock. If your app can recover by itself, do not use one
Readiness	**Removes the pod from Service endpoints** — no restart. This is the one that controls traffic, and the one most workloads actually need
Startup	Disables the other two until it passes. **The correct fix for slow-starting applications**, instead of long liveness delays

The classic outage: an aggressive liveness probe. Under load the app slows, the liveness probe times out, the container is restarted, the remaining replicas absorb more load and also slow, and **the whole service enters a restart cascade** — caused entirely by the probe. Liveness probes should be generous, cheap, and check only that the process is fundamentally alive.

The two rules that prevent most of this. Never make a liveness probe check a dependency — if it queries the database, a database blip restarts every pod in the fleet and turns a degradation into an outage. And **make liveness far more tolerant than readiness**: readiness should shed traffic quickly, liveness should restart reluctantly.

Graceful shutdown, which is genuinely subtle

1. The pod is marked terminating and **removed from endpoints** — but this propagates to every node **asynchronously**
2. `SIGTERM` is sent to the container at the same time
3. After `terminationGracePeriodSeconds` — 30 by default — `SIGKILL` follows

Steps 1 and 2 race, and this causes dropped requests during every deploy. Your pod may receive `SIGTERM` while traffic is still arriving. **The fix is a `preStop` hook that sleeps a few seconds** before shutdown begins, giving endpoint removal time to propagate. Then handle `SIGTERM` by draining in-flight requests.

PodDisruptionBudget caps how many replicas may be voluntarily unavailable at once, protecting you during node drains and upgrades. **Set one for every service that matters** — without it, a routine node drain can take all replicas down simultaneously.

10 · SECURITY

defaults are permissive

Kubernetes defaults are optimised for things working, not for being safe. Out of the box: pods may run as root, network traffic is unrestricted between all pods, Secrets are not encrypted, and containers can often request more privilege than they need. **Every item below is something you must turn on.**

The layers

RBAC	Roles and bindings, namespaced or cluster-wide. **Least privilege, granted to groups.** Audit for wildcards and for anyone holding `cluster-admin`
Pod Security Admission	**PodSecurityPolicy was removed in 1.25.** PSA applies one of three profiles per namespace — `privileged`, `baseline`, `restricted` — in enforce, audit or warn mode. **Aim for restricted**, and use audit mode first to see what would break
securityContext	`runAsNonRoot`, `readOnlyRootFilesystem`, drop all capabilities, `allowPrivilegeEscalation: false`. Cheap, and it closes most container-escape paths
NetworkPolicy	**Default-deny per namespace**, then allow explicitly. The default is allow-all
Admission policy	Kyverno, Gatekeeper or Validating Admission Policy to enforce standards cluster-wide — approved registries, required labels, no privileged pods

Supply chain and identity

- **Scan images** in CI and at admission; **sign them** and verify signatures before they run
- **Pin by digest** rather than by mutable tag. `:latest` in production is an unpinned dependency
- **Disable service account token automounting** where the pod does not call the API — it is on by default
- **Use workload identity** so pods assume cloud roles instead of holding static credentials
- **Enable audit logging** and ship it off-cluster

11 · OBSERVABILITY

what to watch

The stack

- **metrics-server** — the minimum. Powers `kubectl top` and the HPA. **Not a monitoring system** and keeps no history
- **Prometheus** with kube-state-metrics and node-exporter — the standard for cluster and workload metrics
- **Logs go to stdout**; a DaemonSet agent collects and ships them. **Never write logs to a file inside the container** — they vanish with the pod
- **OpenTelemetry** for traces, and increasingly for metrics and logs too
- **Events** are the first place to look in an incident — and they **expire after about an hour**, so ship them somewhere durable

Cluster signals worth alerting on

Pods not Ready	Persisting beyond a rollout window
CrashLoopBackOff	Any occurrence, per workload
Pending pods	Nothing fits — capacity, taints or volume zone constraints
OOMKilled count	Memory limits too low, or a leak
CPU throttling	**Frequently the hidden cause of latency.** Watch it explicitly
Node conditions	Memory, disk and PID pressure; nodes NotReady
API server latency and error rate	Control-plane health
etcd latency and size	Leading indicator of control-plane trouble
Certificate expiry	A boring and genuinely cluster-ending failure
PVC usage	Volumes filling up — silent until writes fail

Alert on symptoms your users feel, not on every Kubernetes condition. A pod restarting is normal; **a service failing its error budget is not.** Cluster-level signals belong on a dashboard for diagnosis; user-facing SLOs belong on the pager.

12 · PACKAGING, DELIVERY AND ROLLOUTS

MID Helm and Kustomize · STAFF GitOps, and safe rollout strategies

Packaging

Raw YAML	Fine for a handful of objects. Duplicates rapidly across environments
Kustomize	**Built into kubectl.** Overlays patch a base per environment. No templating language — what you read is YAML. Ideal for your own applications
Helm	Templated, versioned, parameterised charts with release management. **The standard for distributing third-party software**, and heavier for your own

A common and sensible split: Helm for other people's software, Kustomize for yours. Helm's templating earns its complexity when a chart must serve thousands of unknown users; for your own services, overlays are simpler to read and to debug.

GitOps

- **Git is the desired state**, and an in-cluster agent — Argo CD or Flux — continuously reconciles the cluster toward it
- **The same reconciliation model as Kubernetes itself**, extended to your deployments. That symmetry is why it fits so well
- **Drift is detected and corrected**, so a manual change made during an incident does not silently persist
- **The audit trail is the git history**, and rollback is a revert
- **No cluster credentials in CI** — the agent pulls, rather than CI pushing. A meaningful security improvement

The discipline it demands: stop using `kubectl apply` against production. **Every change goes through git**, or the agent will revert it — which is the point, and it is a real behavioural change for teams used to hot-fixing.

Rollout strategies

RollingUpdate	**The default.** `maxSurge` controls how many extra pods may exist; `maxUnavailable` how many may be down. **Set `maxUnavailable: 0`** for services that must not lose capacity mid-deploy
Recreate	All old pods stop, then new ones start. Downtime, and necessary when versions cannot coexist — a schema change, or a single-writer volume
Blue-green	Two full environments; switch traffic at once. Instant rollback, double the resources
Canary	Route a small share of traffic to the new version and watch. **The safest for user-facing services**, and it needs traffic splitting — from Gateway API or a service mesh

Progressive delivery tools — Argo Rollouts or Flagger — automate canary analysis: shift traffic in steps, watch metrics, and **roll back automatically** when error rate or latency degrades. This converts rollback from a human decision made under pressure into a mechanism.

Making rollouts safe

- **Readiness probes that mean something** — a rolling update trusts them completely to decide the new pod is healthy
- **`minReadySeconds`** so a pod must stay ready before the rollout proceeds. Catches pods that pass a probe then immediately fail
- **PodDisruptionBudgets** so voluntary disruption cannot take everything
- **Backward-compatible changes** — during a rolling update both versions run at once, and they may share a database
- **Watch `kubectl rollout status`** in CI and fail the pipeline on timeout
- **Keep `revisionHistoryLimit`** high enough to roll back to a known-good version

13 · MULTI-TENANCY & QUOTAS

what a namespace is not

A namespace is not a security boundary on its own. It is a scope for names and a place to attach policy. **Without RBAC, network policy, resource quota and pod security applied to it, a namespace isolates nothing** — pods in different namespaces can reach each other freely and compete for the same node resources.

What makes a namespace an actual boundary

- **RBAC** scoped to it, so teams cannot read or change each other's objects — **including Secrets**
- **NetworkPolicy** with default-deny, so traffic does not cross without an explicit rule
- **ResourceQuota** capping total CPU, memory, storage and object counts, so one team cannot consume the cluster
- **LimitRange** setting defaults and maxima per container — **this is how you stop pods with no requests being created at all**
- **Pod Security Admission** label enforcing a profile
- **Separate node pools** with taints, where workloads must not share hardware

LimitRange is the underrated one. It sets a default request on any container that omits one, which quietly eliminates the BestEffort pods that cause most unexplained evictions. It costs one object per namespace.

How far isolation goes

Soft trusted teams	Namespaces with the controls above. **Sufficient for teams within one organisation.** The kernel is still shared
Harder	Dedicated node pools, plus a sandboxed runtime such as gVisor or Kata for stronger workload isolation
Hard untrusted	**Separate clusters.** Virtual-cluster tools give tenants their own API server on shared infrastructure — a genuine middle ground

The honest position: Kubernetes does not offer hard multi-tenancy by default. Tenants share a kernel, a control plane and a network. **For genuinely untrusted workloads, use separate clusters** — and treat the cost as the price of the guarantee.

14 · OPERATIONS — UPGRADES, CAPACITY, COST AND DISASTER RECOVERY

the continuous work of running a cluster, and the parts that are easy to defer until they are urgent

Upgrades — the permanent background task

- **Roughly three releases a year, about fourteen months of support each.** There is no steady state; there is only staying current
- **Upgrade the control plane first, then nodes.** The kubelet may be older than the API server within the supported skew, never newer
- **One minor version at a time.** Skipping is not supported
- **Removed APIs are what actually breaks upgrades.** Check manifests against the deprecation list — `kubectl convert` and tools like Pluto or kubent find them
- **Node upgrades mean draining**, which respects PodDisruptionBudgets — a workload without one can lose every replica at once, and a badly configured one can block the drain forever
- **Rehearse on a non-production cluster** that matches production, including CNI, storage drivers and admission webhooks

The failure to plan for: an upgrade blocked by an add-on. CNI, CSI drivers, the ingress controller and operators all have their own version compatibility. **Check every add-on before starting** — being stranded on an unsupported version because one component has not caught up is a common and painful position.

Disaster recovery

- **etcd backups on a schedule**, stored off-cluster and encrypted — the whole cluster state is in there
- **Test the restore.** An untested backup is a hypothesis, and etcd restore is genuinely fiddly
- **Velero** for namespace and volume-level backup and restore, and for migration between clusters
- **Persistent volumes need their own backups** — snapshotting etcd does not capture your data
- **Be able to rebuild the cluster from git** — with GitOps, cluster loss becomes recreate-and-resync rather than a recovery project. **This is the strongest DR position available**
- **Write down and rehearse**: total cluster loss, control-plane loss, one zone lost

Capacity and reliability

- **Spread nodes across availability zones**, and use topology spread so replicas follow
- **Keep scheduling headroom** so a node loss does not leave pods Pending
- **Control plane HA** — three or five etcd members, several API servers. Managed services handle this
- **Watch pods-per-node limits**, which are usually lower than resource capacity suggests

Cost — where it actually goes

Source	Fix
Over-requested resources	**By far the largest waste.** Requests set from guesswork reserve capacity nobody uses, since the scheduler reserves against requests. Right-size from measurement
Idle nodes	Cluster autoscaler or Karpenter with aggressive consolidation
Wrong instance types	Match workload shape; use spot or preemptible for fault-tolerant work
One load balancer per Service	Consolidate behind an ingress or Gateway
Cross-zone traffic	Real money at volume. **Topology-aware routing keeps traffic in-zone**
Orphaned volumes	StatefulSet volumes outlive the workload deliberately. Audit for unattached disks
Forgotten environments	Namespaces nobody has opened in months

Attribute cost before optimising it. Consistent labels plus a tool such as OpenCost or Kubecost turn a single cluster bill into per-team, per-namespace figures. **Showing teams their own number reduces spend before any technical change** — and the largest line is usually over-requested memory.

15 · PRODUCTION FAILURES — SYMPTOM, CAUSE AND FIX

what actually goes wrong, and the order in which you will meet it

Symptom	Cause	Fix
`Pod Pending`	No node fits — insufficient resources, an untolerated taint, unmet affinity, or a volume in another zone	`kubectl describe pod` states the reason explicitly. Read the events
`CrashLoopBackOff`	The container starts and exits — bad config, a missing dependency, or a failing liveness probe	`logs --previous` for the last attempt. Check whether the probe is the cause
`OOMKilled`	Memory limit exceeded — too low, or a leak	Measure actual usage; raise the limit or fix the leak. **Memory is not compressible**
Latency high, CPU looks low	**CPU throttling from a low limit** — the container is capped even with idle node capacity	Check throttling metrics. Raise or remove the CPU limit; see panel 7
Requests dropped on every deploy	Endpoint removal races SIGTERM	`preStop` sleep; handle SIGTERM by draining; `maxUnavailable: 0`

Symptom	Cause	Fix
Whole service restart cascade	Liveness probe checking a dependency, or too aggressive under load	Make liveness generous and dependency-free; use readiness to shed traffic
All writes to the cluster fail	An admission webhook is down or timing out	Set failure policy deliberately; exclude critical namespaces; tight timeouts
Everything broken in odd ways	CoreDNS unhealthy or under-resourced	Check CoreDNS first when symptoms are diffuse. Give it resources and replicas
Node drain hangs forever	A PodDisruptionBudget that can never be satisfied	Review PDBs against replica counts — one replica with `minAvailable: 1` blocks every drain
Upgrade blocked or breaks apps	Removed APIs, or an add-on not yet compatible	Scan manifests for deprecated APIs; check every add-on before starting
Random pods killed on a busy node	BestEffort pods with no requests, evicted first	**Set requests on everything**; add a LimitRange per namespace

What breaks first

#	Limit
1	**Missing or wrong resource requests.** The root of evictions, Pending pods, throttling and most cost waste — and the cheapest to fix
2	**Misconfigured probes**, which turn a degradation into a restart cascade
3	**Upgrades deferred** until the version is unsupported and add-ons have moved on
4	**etcd or control-plane health**, usually disk latency
5	**Raw cluster scale** — the thing people design for, and the last to bind

Note the order. Nothing in the top three is about scale or architecture. They are numbers in a manifest, probe configuration and maintenance discipline — all visible in advance, and all fixable without new infrastructure.

Degradation

- **API server down** — running pods keep running. You lose all control, scaling and self-healing. **Serious but not immediately user-facing**
- **etcd down** — the control plane is read-only at best. Workloads continue; nothing can change
- **One node lost** — pods are rescheduled if capacity exists. **This is the case Kubernetes handles best**
- **A zone lost** — survivable only if nodes and replicas were spread across zones. Verify this before you need it
- **CNI or CoreDNS degraded** — the most confusing outage available, because everything fails oddly rather than cleanly

16 · PRODUCTION CHECKLIST

before workloads depend on it

- ☐ **Resource requests set on every container**
- ☐ **Memory limits set**; CPU limit policy decided deliberately
- ☐ **LimitRange per namespace** so nothing lands as BestEffort
- ☐ **Requests right-sized from measurement**, not guesses
- ☐ **QoS class understood** for every critical workload
- ☐ **Readiness probes on everything serving traffic**
- ☐ **Liveness probes generous and dependency-free**, or absent
- ☐ **Startup probes for slow starters**
- ☐ **`preStop` hook and SIGTERM handling** for graceful shutdown
- ☐ **PodDisruptionBudget per service**, satisfiable at current replica count
- ☐ **Topology spread across zones and nodes**
- ☐ **Anti-affinity or spread** so replicas never share one node
- ☐ **`maxUnavailable: 0`** where capacity must not drop
- ☐ **`minReadySeconds` set** on critical rollouts
- ☐ **Images pinned by digest**; no `:latest`
- ☐ **Images scanned and signed**; signatures verified at admission
- ☐ **Pod Security Admission enforced**, aiming at restricted
- ☐ **`securityContext` set** — non-root, read-only root, capabilities dropped
- ☐ **Default-deny NetworkPolicy per namespace**; egress considered
- ☐ **CNI verified to enforce NetworkPolicy**
- ☐ **RBAC least-privilege**; wildcards and cluster-admin audited
- ☐ **Service account token automounting disabled** where unused
- ☐ **etcd encryption at rest enabled**, ideally with KMS
- ☐ **Secrets from an external store**, never committed to git
- ☐ **Workload identity instead of static cloud credentials**
- ☐ **Admission webhook failure policy and timeouts set**
- ☐ **Critical namespaces excluded from webhooks**
- ☐ **Ingress controller actively maintained** — Ingress NGINX users have a migration to plan
- ☐ **Gateway API considered** for new edge work
- ☐ **CoreDNS resourced, replicated and monitored**
- ☐ **StorageClass reclaim policy understood** before deleting anything
- ☐ **Persistent volumes backed up separately** from etcd
- ☐ **etcd backed up off-cluster and a restore tested**
- ☐ **Cluster rebuildable from git**
- ☐ **Upgrade cadence planned**; deprecated APIs scanned for
- ☐ **Add-on compatibility checked** before every upgrade
- ☐ **Metrics, logs and events shipped off-cluster**
- ☐ **Alerting on user-facing SLOs**, not on every pod restart
- ☐ **CPU throttling monitored explicitly**
- ☐ **Cost attributed per team via consistent labels**

The one that summarises the sheet: **state whether every container has resource requests, what happens to a pod when its liveness probe fails under load, what the network policy and pod security defaults are in each namespace, when etcd was last restored from backup, and how the next version upgrade will be performed.** Those five answers cover almost every incident this platform will hand you — and none of them are about how many nodes it can run.